

# EL-BA2 – Projet ICC – Brick Breaker

Nadav Solomon - 398747

Ilhan Salvatore Legio - 397526

## Rendu 2 – activités individuelles:

Pour le 2<sup>ème</sup> rendu, les activités individuelles ont été partagés de la façon suivante :

**Nadav** a travaillé sur l'architecture, lancement du jeu, lancement sur macOS, affichage de la fenêtre.

**Ilhan** a travaillé sur les boutons, déplacement de la raquette, affichage des objets, ajout de balle.

### **1. Hiérarchie des classes de briques.**

Super-classe abstraite 'Brick' :

- Attributs gérés : 'bounds\_' (type `tools::Square`) définissant la géométrie et 'type\_' (int) identifiant la nature de la brique.
- Méthodes communes : 'check()' vérifie la validité de la taille et la position dans l'arène par rapport à 'arena\_size'.

Classes dérivées (RainbowBrick, BallBrick, SplitBrick) :

- Introduit l'attribut spécifique pour RainbowBrick 'hit\_points\_' (vie).

Polymorphisme et Virtualité :

- Méthodes Virtuelles Pures : 'draw()' et 'write()' (dans 'Brick', forçant chaque sous-classe à définir sa propre représentation graphique et son format de sauvegarde.) !
- 'RainbowBrick' substitue la méthode 'check()' pour y ajouter une vérification du domaine de validité des points de vie.

### **2. Structuration des données dans « game ».**

Attributs de la classe 'Game' :

- Variables d'état : 'score\_' et 'lives\_' (entiers).
- Entités : 'paddle\_' (Raquette) et 'balls\_' (`std::vector` de Ball).

Mémorisation des ensembles :

- Les briques sont stockées dans un '`std::vector<std::unique_ptr<Brick>>`'.
- L'usage de '`unique_ptr`' garantit la gestion automatique de la mémoire et permet le traitement polymorphique de la collection hétérogène.

### **3. Description du modèle « tools ».**

Types définis (Structures) :

- Point : Structure simple gérant des coordonnées (x, y) avec surcharge de l'opérateur + pour les déplacements.
- Square / Circle : Modèles de données pour les briques (carrées) et la balle/raquette (circulaires), incluant des méthodes de dessin déléguées au module 'graphic'.

Utilitaires :

- Détection d'intersections : Fonctions 'intersects\_...' gérant les collisions entre cercles et carrés avec une marge d'erreur 'epsil\_zero' (`constexpr`).
- Intégrité : Fonctions 'is\_...\_in\_...' vérifiant que les entités restent dans les limites de l'arène de jeu.